

Coordinating Multiple Traffic Lights with Fog Computing and V2X Communications for Efficient Traffic Management

Samuel Arantes
Centro ALGORITMI
Universidade do Minho
Braga, Portugal
pg54218@alunos.uminho.pt

Maria João Nicolau
Centro ALGORITMI
Universidade do Minho
Braga, Portugal
joao@dsi.uminho.pt

Marcelo Alves
Bosch Car Multimedia
Bosch
Braga, Portugal
Marcelo.Alves2@pt.bosch.com

António Costa
Centro ALGORITMI
Universidade do Minho
Braga, Portugal
costa@di.uminho.pt

Abstract—The global increase in the number of vehicles has exacerbated problems like traffic congestion, long travel times, harmful emissions and fossil fuel consumption. Traditional traffic control cannot deal with this increasing traffic density and its dynamic variations throughout the day. To address this challenge, we propose a Smart Traffic Light (STL) algorithm that dynamically adjusts the phases of traffic lights at intersections based on real-time traffic density. The system is based on the Fog Computing paradigm and uses Roadside Units (RSUs) to collect vehicle data and map intersections. This information is shared with a Fog Node that dynamically determines the optimal traffic light phases for the intersection within its control area. Additionally, the system enables coordination between adjacent intersections to improve traffic flow efficiency throughout its actuation area. The proposed approach was examined using the Eclipse MOSAIC framework with the SUMO mobility simulator to evaluate the impact of dynamic traffic light control and intersection coordination. Under the specific conditions and parameters tested, the results demonstrate that the STL algorithm significantly reduces average trip duration, waiting times, emissions, and fuel consumption compared to static traffic light systems. In the specific simulation setup used, enabling intersection coordination further improved efficiency — reducing trip duration by 17%, waiting times by 30%, and emissions and fuel consumption by approximately 17%. These values reflect the tested scenario and may vary with different road geometries or traffic conditions.

Index Terms—Vehicular Networks, V2X, Traffic Control, Fog Computing

I. INTRODUCTION

Traffic congestion remains a significant challenge in urban areas, causing substantial time loss, economic costs, and environmental impact. According to the 2023 INRIX Global Traffic Scorecard [1], drivers in New York (United States) lost an average of 101 hours to congestion, while those in London (United Kingdom) lost 99 hours. Moreover, intersections are critical points of traffic flow, where approximately one-quarter of traffic fatalities and half of all traffic injuries occur annually in the United States [2]. Addressing these issues is crucial, as improving traffic efficiency reduces time loss and transportation costs and contributes to a cleaner environment. Numerous studies [3], [4] have explored ways to improve

intersectional traffic flow by reducing travel duration, waiting times, and emissions. Traditional approaches, such as pre-timed traffic light systems or centralized control, often lack the flexibility to adapt to real-time traffic conditions. Recent advancements have introduced dynamic traffic light control systems that leverage machine learning, the Internet of Things (IoT) and Vehicular ad hoc networks (VANETs). While most existing traffic light systems effectively optimize individual intersections, achieving seamless coordination across adjacent junctions remains a challenge, particularly in scaling real-time adaptability and maintaining synchronized flow within large urban networks [4]. To tackle these challenges, we propose a Fog-based STL algorithm designed to enhance traffic management across multiple intersections at a larger scope. The system leverages Fog computing, RSUs and vehicles equipped with Vehicle-to-Everything (V2X) communication devices and sensors. A Local Dynamic Map (LDM) is generated at the Fog Node using data collected from RSUs, providing a comprehensive view of the area. This enables dynamic control of traffic lights and real-time updates for vehicles, ensuring smoother traffic flow. By incorporating a larger awareness area, the proposed solution achieves superior traffic management and reduces waiting times, travel duration, and emissions.

The remainder of this paper is structured as follows: Section II presents the related work. Section III details the proposed approach, identifying the system components, communication protocols, and control algorithms. Section IV presents the simulation setup, and Section V analyses and discusses the results obtained through simulation. The paper ends with conclusions and future work in Section VI.

II. RELATED WORK

Various academic research works have been conducted in the field of VANETs to enhance the efficiency of current Traffic Lights Systems by reducing vehicle waiting times. Typically they explore two types of architectures, either infrastructure-based using Vehicle-to-infrastructure (V2I) communications with RSUs and/or Fog Nodes or decentralized architecture with V2X communication only between vehicles.

In this section, we detail several works related to the approach proposed in this paper.

A. Smart Traffic Lights

Xiao et al. [5] proposed an Intelligent Traffic Light (ITL) approach to reduce vehicle CO₂ emissions based on VANETs. The information sent by approaching vehicles is captured by RSUs placed at each intersection. This data is then used to dynamically adjust the Traffic Light (TL) phases based on various factors, including traffic density, average waiting time per lane, queue length and special conditions such as the presence of emergency vehicles. The algorithm works by first checking for any special priority conditions, then evaluating whether any lane's average waiting time exceeds a threshold, followed by comparing queue lengths to prioritize the lane with the longest waiting time, then assessing traffic density to favour the most congested direction, and finally defaulting to the current phase if none of the other conditions are met, thereby ensuring an efficient, fair, and adaptive traffic light control. When comparing with traditional pre-timed control method their proposed scheme can effectively reduce vehicle's average waiting time, stop times and CO₂ emissions.

The authors of RoadRunner+ [6], proposed a Autonomous Intersection Management (AIM) system in order to improve safety and road efficiency integrating Connected Autonomous Vehicles (CAV), pedestrians and neighbouring intersections. When CAVs approach an intersection AIM schedules their crossing in a collision-free manner while minimizing their waiting times. Additionally, this system assigns a lane and an optimal speed to cross the intersection to minimize stopping and save fuel. Furthermore, it detects congestion in the exits of the intersections to prevent spillback and diverts incoming traffic to minimize it. As for pedestrians, they can send a crossing request, using a Smartphone, to the AIM who will reserve a safe time-slot for them. This system was evaluated in the SUMO microscopic simulator and results showed that RoadRunner+ achieves an higher throughput, less travelling delay and fuel consumption when compared to other AIM systems. Also, all collisions caused by spillback were prevented.

B. Fog-Based Smart Traffic Light control

Gamel et al. [7], proposed a Fog-Based Traffic Light Management Strategy (TLMS) using a fuzzy inference engine. This strategy calculates optimal lane assignments for vehicles approaching the intersection, thereby reducing the average waiting time for stopped vehicles. Vehicles send their information to a nearby TL where a vehicle list will be created containing all the information, which is then relayed to a Fog Node responsible for managing this area of the network. Based on the fuzzy inference engine, the Fog Node makes a decision and the road status (open or closed) is sent back to the TLs who broadcast this information to the vehicles. When comparing their strategy with others available, TLMS achieved better results in terms of average delay per vehicle, fuel consumption, and intersection throughput.

Qin and Zhang [8] proposed a multi-agent distributed Intelligent Traffic Light (ITL) control method based on the fog computing platform and the Q learning algorithm, a reinforcement learning technique, to reduce vehicles delay time and retention rates. In their architecture, whenever a vehicle enters the communication range of a Fog Node and a Roadside Unit (RSU), it transmits its current position, speed, and driving direction. The Fog Node responds with the current traffic light (TL) status and forwards the traffic flow (TF) data to the TL agent. The agent then uses the Q-learning-based control algorithm to determine and update the optimal TL timing scheme. Fog Nodes also share TF data across intersections, coordinate local Fog Network activity, and aggregate regional TF data for upload to the cloud. While fog computing handles local traffic optimization, the cloud layer is responsible for managing traffic across the city-wide network. Additionally, collected TF data are archived as historical input for continuous learning and system improvement. The TF data collected is stored and used as historical data to the proposed TL algorithm. The proposed method was tested using the VISSIM-EXCEL VBA-MATLAB platform. Results showed that the system can effectively reduce the delay time and retention rate of vehicles.

III. PROPOSED APPROACH

The main goal of the proposed system is to manage and control TLs in an intersection, using a STL algorithm, and to extend this coordination to regulate multiple adjacent or non-adjacent intersections across a broader area. This implementation targets the reduction of trip duration and waiting time for vehicles, as well as lowering pollution by reducing CO₂ emissions and fuel consumption. In order to enable coordination between adjacent intersections, we developed a system capable of mapping the area, aggregating this data and transmitting it to a central controller who has an overview of the actuation area and can take into account all the information in order to decide the optimal TL phases. To map an intersection and aggregate the information received from vehicles we opted for an RSU, which is an infrastructure with V2X communication capabilities and sufficient computational power to handle the required tasks. The RSU will serve as a bridge for the information in a single intersection to be transmitted to the higher layers where all the intersections information is known. For this purpose we opted to use a Fog Node to aggregate all the data and determine the TL phases. This node has higher computational power, enabling more complex algorithms for determining the globally optimal TL phases to be assigned for all RSUs and their assigned physical TLs.

This implementation is based on the Fog computing paradigm, as evidenced in Figure 1, which provides a layered view of the architecture. The system is divided into two layers, the edge layer and the fog layer. Our system enables the collection of real time traffic information at the edge layer, with data aggregation and decision making being delegated to the fog layer.

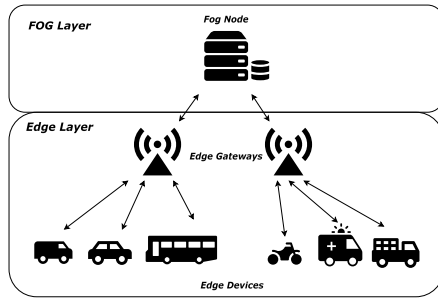


Fig. 1. Proposed system architecture consisting of two layers: the edge layer, which includes vehicles and edge gateways, and the fog layer, consisting of a Fog node.

This system is composed of three main entities :

1) *Edge devices*: Edge devices are any type of vehicle equipped with an Application Unit (AU) and an On-Board Unit (OBU) capable of V2X communication. Using the intra-vehicle sensors and a Global Positioning System (GPS) with lane-level accuracy, vehicles periodically broadcast ego information, via CAM messages, to every entity in range. They are an integral part of the system, designed to navigate intersections efficiently while complying with the TL phases. Each device consists of two primary modules: a communication module and a sensing module.

- **Communication module**: responsible for exchanging V2X compliant messages. Vehicle update messages transmission interval should be between 100 and 1000 ms [9].
- **Sensing module**: responsible for gathering data from the vehicle's intra-sensors. This module must be capable of extracting the vehicle's position within the sub-meter range with low latency to ensure constant updates on the vehicles location. Also it must collect and process raw data coming from the vehicle sensors regarding their speed, acceleration, heading and blinker status.

2) *Edge Gateways*: Edge Gateways are RSUs strategically positioned at intersections with the job of mapping the intersection regarding the dynamic traffic elements. This is done by creating an LDM and updating it every time new information is received. This map contains the current state of the intersection including vehicle positions on lanes and related information. They are composed of two modules: a communication module and a mapping module.

- **Communication module**: responsible for exchanging V2X compliant messages. This module must continuously share the mapping information with the Fog Node and transmit TL state messages to the vehicles in its assigned intersection.
- **Mapping module**: responsible for keeping an updated LDM and sharing it with the Fog Node. This module must generate a contextual layer containing dynamic information, such as the list of vehicles per lane along with each vehicle's details. It must generate and update the map using at least one-hop received data and refresh the map whenever new information is received. Additionally, this

module is responsible for informing the Fog Node about the geometry of its assigned intersection, using MAPEM (MAP Extended Message) messages during initialisation.

3) *Fog Node*: The Fog Node is held in the Fog Layer and is the main piece of this architecture. It is responsible for managing and controlling the intersections within its actuation area. For this it also creates and keeps an updated LDM of the actuation area which is a result of the merging of all the LDMs received from Edge Gateways. Using the information contained in the LDM regarding intersections states and vehicle information it is capable of deciding on the TL state for each intersection in the actuation area. The Fog Node is composed of three modules: a Communication module, a Mapping module and a Traffic Management module.

- **Communication module**: responsible for exchanging V2X compliant messages. Receives Mapping information from Edge Gateways and transmit the TLs decisions.
- **Mapping module**: responsible for maintaining an updated LDM by merging the individual LDMs of all intersections within the actuation area. This module must generate a contextual layer that includes static infrastructure data (e.g., lane geometry, road layout, directions), and dynamic environmental data such as the list of vehicles per lane and their information. During initialisation, this module also registers every RSU in the actuation area by saving the intersection geometry, including all ingress lanes, egress lanes, and directions. This information is used to ensure the LDM is always up-to-date and can be utilized by other modules as needed.
- **Traffic Management module**: responsible for running the TL algorithm for each intersection using the information contained in the LDM.

A. Communication Protocol

To ensure compatibility with other academic research we chose to follow the ETSI guidelines for V2X and for that purpose, as mentioned in the previous sections, we use the following message types:

- CAM - Cooperative Awareness Message
- MAPEM - MAP Extended Message
- SPATEM - Signal Phase And Timing Extended Message

As stated in [10] CAM messages are exchanged between C-ITS stations equipped with V2X technology to create and maintain awareness of each other and to support cooperative performance of vehicles using the road network.

These messages provide information regarding the presence, position, dynamics, and basic attributes of the originating station. According to the European Telecommunications Standards Institute (ETSI) EN 302 637-2 standard, CAM messages should be generated at intervals no shorter than 100 ms (10 Hz) and no longer than 1000 ms (1 Hz). For RSUs this interval should not be greater than 1000 ms (1 Hz). CAM messages generation is triggered by some conditions:

- The difference of the current heading of the originating ITS-S and the heading in the previous CAM exceeds 4°.

- The distance between the current position and the position in the previous CAM exceeds 4m.
- The difference between the current speed and the speed in the previous CAM exceeds 0.5 m/s.

Typically, these update messages do not occur at fixed intervals. However, for simplification, we opted for the worst case update scenario with a periodicity of 1000 ms.

During the initialisation process, RSUs share the geometry of their assigned intersection to the Fog Node via the MAPEM message. This message enables us to share details about intersections, road segments, and user restrictions. Specifically, our goal is to provide information about the lanes, including their entry and exit points, connections, directions, and possible manoeuvres. This message will be sent once when the RSU is activated or whenever changes occur to the road segments.

To transmit the TL phases, we use SPATEM messages, which convey the current status of one or more signalized intersections. These messages are generated by the Fog Node and sent to the relevant RSU, which then broadcasts it to the vehicles at the intersection. SPATEM messages are transmitted at a periodicity of 1000 ms.

Regarding the update messages for the LDM originating in the RSU to the Fog Node we could not identify a standard message that fully met our requirements so we decided to use a custom message (UPDATE). This message includes content from the MAPEM message augmented with additional fields containing vehicles information. It will be transmitted with a periodicity of 1000 ms.

B. Algorithms

For this work, we developed a STL algorithm capable of dynamically determining the TL phases at an intersection based on their current state regarding traffic flow. Furthermore, we extended the perception of this algorithm to enable a broader understanding of an actuation area and to dynamically decide on the various intersection with TLs taking into account the state of adjacent ones. Additionally, as baseline, a static pre-timed version was developed, corresponding to the functionality of most current traffic lights. By comparing this approach with a dynamic traffic light control system, we aim to understand the effects of enabling adaptive control of TLs at intersections and how that will affect the vehicles trip. In addition to that, we intend to experiment with a central controller capable of managing multiple TLs in an actuation area having knowledge of all the intersections states and taking into account adjacent ones to optimize the traffic flow. The following provides a detailed description of the three algorithms developed for this study, each addressing a different aspect of traffic light control:

1) *Static version*: This algorithm will serve as baseline for all the test phases performed in this work. With this type of algorithm, we pretend to simulate the current state of the majority of intersections ruled by TLs, where the decision is pre-timed: 27 seconds green, followed by 3 seconds of yellow, and then a red phase while the green + yellow phase rotates sequentially across the other lanes in the intersection.

This version does not account for the current state of the intersection as the decision is predetermined. These values are parametrized and were chosen for this specific case based on the scenario.

2) *STL algorithm with coordination*: The main goal of this algorithm is coordination between adjacent intersections in order to optimize traffic flow and mitigate the effect of spillback incidents. Spillback is the blockage of an intersection by traffic as a result of the TLs changing and preventing progress. The algorithm is divided into five phases:

- **Registration phase**: during start-up, the RSU transmits the intersection geometry, its identifier and intersection coordinates to the Fog Node, which registers both the RSU and the geometry, including all lanes and possible directions.
- **Detection phase**: whenever a vehicle message is received within the detection zone of an RSU, its information is added to the LDM.
- **Update phase**: with a fixed interval the information present in the LDM is shared from the RSU to the Fog Node. The Fog Node updates its LDM, which is a result of merging all LDMs in the actuation area.
- **Decision phase**: the Fog Node uses the information in the LDM to decide on the TL states for the intersections.
- **Warning phase**: the physical TL is changed by the Fog Node and the decision is passed into the RSU, which broadcasts it to vehicles.

The STL algorithm is used in the decision phase to decide the TL phases of the intersections. For an easier understanding of how this algorithm works let's define LENT as an ingress lane of an intersection and LEXT as an egress lane. In figure 2 we can visualize an example of an ingress and egress lane of an intersection.

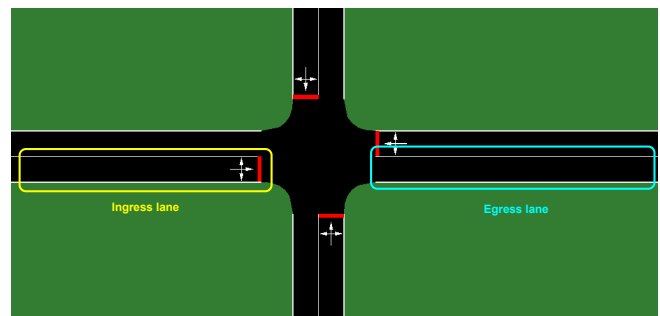


Fig. 2. Example of an ingress and egress lane of an intersection.

On every RSU registration, the Fog Node saves lists containing the LENT, LEXT, directions and Spill Free and Can Spill lists. Spill Free list contains the ingress lanes of an intersection which all of the possible egress connections will not lead to an adjacent intersection, and the Can Spill list the exiting lanes of an intersection that lead to an adjacent intersection. Whenever an RSU is registered its lanes are checked to see if there is any match of LENT with LEXT in other intersection already registered. If there is a match it means that these intersections

are adjacent and we have to take into account the number of vehicles we send to the lanes in common to prevent spillback. For this, the LENT who is a LEXT on another intersection is placed in the Spill Free list because it will never cause spillback considering that the vehicles on that lane are coming from the adjacent intersection and have no way of going there again from the current intersection. When we have a match on LENT with LEXT, we also have the opposite LEXT matches with an LENT and so this lane is added to the Can Spill list because vehicles exiting through this LEXT enter the adjacent intersection. If that lane becomes congested, it may cause spillback. To help visualize this, figure 3 shows an example of common lanes between two adjacent intersection, where the ingress lanes in the yellow square would be the Spill Free lane in their respective intersection and the remaining ingress lanes on the Can Spill list. Additionally, for every RSU there

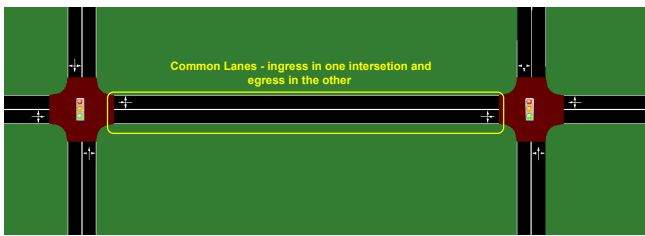


Fig. 3. Example of commons lanes between two adjacent intersections.

is a list of available lanes which includes all LENTs of an intersection and are the lanes available to be placed in the green phase. To avoid starvation, once a lane gets a green light, it is removed from the available lanes list. This continues until all lanes have been selected once. Only then is the list refilled, ensuring that every lane gets a green light at least once before any lane is selected again. The exception to this is the Spill Free lane since in the event we cannot place any lane as green due to congestion in the adjacent intersection this lane can be placed as green by the reason it will not cause spillback. The algorithm runs periodically according to the update messages with an interval of 1000ms and goes through the following steps:

- **Increment current phase timer:** Each phase can last up to 30 seconds (27 seconds green + 3 seconds yellow). These values are parametrized and were chosen for this specific case based on the scenario.
- **Check the available lanes list:** If it is empty, populate it with all ingress lanes.
- **Check current phase:**
 - If the timer exceeds 27 seconds, switch to yellow phase.
 - If the timer reaches 30 seconds or the current green lane becomes empty before that, switch to red and select new lane.
- **If new lane needs to be selected:**
 - **If in Spill Back State:** Select the Spill Free lane as next phase to help free up space and resolve the issue.

– If not in Spill Back State:

- * Sort available lanes in descending order by vehicle count.
- * Check, in order, for each lane, until a suitable lane is found:
 - If the total number of vehicles in the common lane plus vehicles in queue wanting to exit through the common lane (checked using vehicle blinkers and intersection directions) is more than the maximum permitted (based on the distance between the intersection centre coordinates), the lane cannot be chosen; otherwise, choose the lane.
 - If it's more than the maximum permitted, check just the first maximum permitted vehicles. If they can proceed without congestion, select the lane and calculate the green phase duration dynamically, to limit the crossing to this number of vehicles, based on: the last vehicle distance to the centre of the intersection, acceleration, road maximum speed (50 km/h), the number of cars in front and a delay for each car in front (1.5 s). If they can't skip it.
- * If no lane is suitable:
 - If a Spill Free lane exists, select it and mark the adjacent intersection as in Spill Back state. This ensures the adjacent intersection will prioritize the shared lane next cycle.
 - If there are multiple neighbouring intersections and no Spill Free lane, all lanes stay red until one neighbouring intersection becomes uncongested and a lane can proceed. In this type of intersections whenever a lane is selected it is not removed from the available lanes list.

3) *STL algorithm without coordination:* This algorithm is the same as the previous one, but the verification steps are skipped, meaning only the information from the intersection is taken in account when deciding the TLs phases. The Fog Node, when deciding on the TLs, only accounts for the LDM of an intersection, meaning there is no broader knowledge being taken into account. This results in a round-robin process where the lane with the most vehicles in the available lanes list is always be chosen.

IV. SIMULATION SETUP

Due to the resource constraints that make developing and implementing this use case in a real-world prototype infeasible, simulation tools were chosen as the preferred approach for its evaluation. To accurately model the behaviour of the traffic management system and evaluate the performance of the various algorithms, we required simulators that could effectively mimic real-world traffic scenarios, including the interaction between vehicles, RSUs and the Fog Node. The choice for simulators was done by analysing the various possible simulation environments. The most typical environments are coupled simulators using an overall framework capable of giving the support and integration between all of them

into one simulation. For the framework the choice fell on the Eclipse Mosaic Framework. Eclipse Mosaic [11] is a multi-scale simulation framework in the field of smart and connected mobility. It allows coupling simulators from various domains towards a comprehensive simulation tool. Data exchange and time management is implemented by the Runtime Infrastructure (RTI), which is the heart of MOSAIC. Simulation models are coupled to the RTI using High Level Architecture (HLA) inspired interfaces. Each simulator is wrapped into a "Federate" object which is linked to an "Ambassador" which is directly coupled with the RTI. We selected SUMO [12] as the mobility simulator because it supports the representation of complex urban scenarios with traffic lights and road users, and provides various output files for extracting key metrics during the evaluation process. For the network simulator, we opted for Mosaic SNS, a simple simulator that allows us to exchange V2X-messages through ad-hoc communications [13]. Figure 4 presents a snapshot of the road map used for all the test phase on this work. This road map was created using the software netedit [14] where we designed a scenario to mimic an urban context with adjacent intersections and with different geometries (4 way and 3 way) where adjacent ones are placed within 100 meters of each other. Intersections with a TL are ruled by the algorithms and have a RSU deployed on it while the remaining are ruled by right-of-way priority type at intersections where vehicles on low-priority roads must wait until all vehicles on high-priority roads have cleared the intersection. The central controller, the Fog Node, is positioned in a way of being capable of communicating with every RSU. Vehicles, RSUs and the Fog Node are all running applications developed using the JAVA language. The algorithm is present in the Fog Node application where using the information retrieved from the various RSUs is capable of deciding on each TL phase and pass the decisions onto the RSUs. RSUs have full knowledge of the geometry of their assigned intersection and map the state of each ingress lane using the information transmitted from vehicles within the RSU range (100 meters). Vehicle messages that are on ingress lanes of other intersections or are outside the range are discarded. The vehicle application is responsible for gathering the raw data from sensors and broadcasting it to the in range RSUs. Additionally, they receive the SPATEM messages with the TL states. Eclipse Mosaic enables fine-grained scenario configuration using a set of configuration files. The "mapping config" file is of key importance, as it handles the definition, parametrization and application assignment for all entities in the simulation. In this file we specify every infrastructure present in the simulation (RSUs and Fog Node) setting their coordinates and application. As for the vehicles configuration we created a prototype "Vehicle" to define the overall parameters for every vehicle in the simulations. Furthermore, we created vehicle generators for each route in the simulation. These generators allow us to configure the number of vehicles for that route, the starting time of the spawning and the density of vehicles per hour. A total of 27 vehicle generators were created, each with a different route assigned to it. Each route

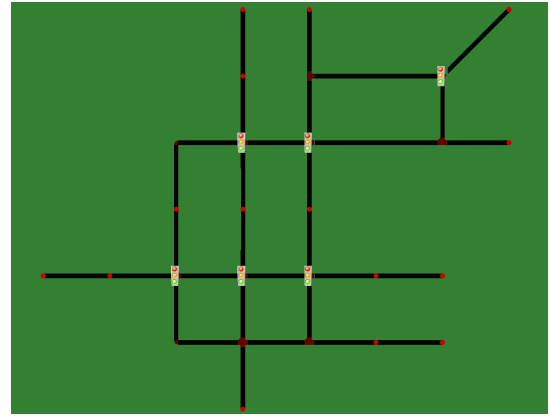


Fig. 4. Simulation road map on netedit, showing five TL ruled intersections with deployed RSUs and four ruled by right-of-way priority.

has different start and end positions, and may include one or more crossings in its path. Regarding the network simulator, Mosaic SNS, only a few changes were made to the default configuration from the one provided by Eclipse Mosaic. This configuration defines various parameters related to message transmission in an ad-hoc network, including maximum hop limits, delay characteristics, and transmission settings. The Maximum TTL limits the number of hops a message can take, while the Single-hop Radius sets the fallback range for direct communication. The Ad-hoc Transmission Model specifies how ad-hoc transmissions are handled. It includes a Type that defines the model and a Multi-hop Delay configuration that determines the minimum and expected delay. The Multi-hop Transmission settings define the probability of message loss (Loss Probability) and the maximum number of retries allowed (Maximum Retries). For direct communication, the Single-hop Delay model defines a delay distribution with a specified Type, Minimum Delay, Maximum Delay, and the number of Steps in the distribution. The Single-hop Transmission settings includes the probability of loss (Loss Probability) and the number of allowed retries (Maximum Retries). Table I shows the values for each of the configuration parameters.

Although the maximum number of hops was configured to 20, each message in the simulation was set to only broadcast to a maximum of 2 hops.

V. TEST AND RESULTS

The primary objective of this work is to evaluate the benefit of enabling coordination between adjacent intersection to optimize the overall traffic flow. For that, the following performance metrics were tracked:

- Average trip duration per vehicle (seconds).
- Average waiting time per vehicle (seconds).
- Average CO2 emissions per vehicle (grams).
- Average fuel consumption per vehicle (grams).

For the simulations we defined two different traffic scenarios where we vary the density of vehicles per hour entering the scenario. For this, we changed the density of vehicles in every generator for each simulation. For low traffic simulation we

TABLE I
PARAMETERS AND RESPECTIVE VALUES FOR THE NETWORK SIMULATOR CONFIGURATION.

Parameter	Value
maximumTtl	20
singlehopRadius	509.4 m
singlehopDelay.type	SimpleRandomDelay
singlehopDelay.minDelay	400000 ns
singlehopDelay.maxDelay	2400000 ns
singlehopDelay.steps	5
singlehopTransmission.lossProbability	0.0
singlehopTransmission.maxRetries	0
adhocTransmissionModel.type	SimpleAdhoc
MultihopDelay.type	GammaDelay
MultihopDelay.minDelay	10 ms
MultihopDelay.expDelay	30 ms
MultihopTransmission.lossProbability	0.0
MultihopTransmission.maxRetries	2

set a density of 50 vehicles per hour and the high traffic as 125 on each of the routes. As for the number of vehicles in the simulation we used a *Python* script to randomly assign a number of vehicles and a starting time to a route. To ensure a realistic and balanced simulation setup, the starting time of the first vehicle on each route was randomized between 0 and 30 seconds. Additionally, the number of vehicles per route was randomized between 35 and 65, resulting in ten distinct configuration sets. Each configuration set was used to test every algorithm under both traffic density conditions. On average, each simulation included approximately 1369 vehicles. In low traffic scenarios, due to reduced vehicle flow, this number averaged around 1230 vehicles. In total, we conducted 60 simulations. We used the same 10 configuration sets to evaluate each of the three algorithms under both Low and High traffic densities. Specifically, for each traffic density, we ran 10 simulations with each one of the algorithms.

To obtain all the metrics and analyse the results we used Sumo output files. Table II presents the results of all metrics we wanted to evaluate showing, for each one, the average values of the ten simulation of each traffic scenario and algorithm plus the standard deviation. Table III shows the comparison in percentage of the values obtained in the simulations of both dynamic versions compared with the Static Version, where the percentage represents the reduction in the respective metric.

A. Low traffic results

The results indicate that in this scenario both the dynamic versions outperform the static version by some margin, this is because vehicles may need to wait in a red light even if they are the only vehicle in the intersection, leading to worse traffic flow. In this scenario both dynamic versions operate in a very similar way as there is not enough traffic to demonstrate the value of adjacent TL coordination. The presented results

TABLE II
RESULTS OBTAINED IN THE SIMULATIONS FOR EACH OF THE DEFINED METRICS, INCLUDING THE ASSOCIATED STANDARD DEVIATION VALUES FOR BETTER UNDERSTANDING OF THE VARIABILITY AND CONSISTENCY ACROSS DIFFERENT ALGORITHMS AND TRAFFIC DENSITIES.

	Traffic density	Algorithm		
		Static Version	STL	STL w/ coordination
Average trip duration (seconds)	Low	129.92 $\pm$ 1.76	57.48 $\pm$ 1.02	56.66 $\pm$ 1.01
	High	253.56 $\pm$ 6.41	146.10 $\pm$ 7.9	120.12 $\pm$ 9.72
Average waiting time (seconds)	Low	67.56 $\pm$ 1.52	1.98 $\pm$ 0.77	1.20 $\pm$ 0.65
	High	181.65 $\pm$ 5.61	74.37 $\pm$ 7	51.73 $\pm$ 8
Average CO2 emissions (grams)	Low	360.27 $\pm$ 4.88	158.57 $\pm$ 3.37	156.35 $\pm$ 3.05
	High	714.78 $\pm$ 16.56	406.03 $\pm$ 19.18	335.84 $\pm$ 25.88
Average fuel consumption (grams)	Low	114.91 $\pm$ 1.55	50.57 $\pm$ 1.07	49.87 $\pm$ 0.95
	High	227.99 $\pm$ 5.28	129.51 $\pm$ 6.12	107.12 $\pm$ 8.25

TABLE III
COMPARISON OF THE RESULTS OBTAINED IN THE SIMULATIONS OF THE DYNAMIC ALGORITHMS VERSUS THE STATIC VERSION (THE PERCENTAGES REPRESENT THE REDUCTION IN THE ANALYSED METRICS FOR THE DYNAMIC ALGORITHMS COMPARED TO THE STATIC VERSION).

	Traffic density	Algorithm vs Static Version	
		STL	STL w/ coordination
Average trip duration (seconds)	Low	56%	56%
	High	45%	54%
Average waiting time (seconds)	Low	97%	98%
	High	59%	72%
Average CO2 emissions (grams)	Low	56%	57%
	High	43%	53%
Average fuel consumption (grams)	Low	56%	57%
	High	43%	53%

prove that enabling dynamic control of traffic lights leads to a reduction of 56% on trip duration and 98% of the waiting time needed. Emissions and fuel consumption are reduced by around 57%.

B. High traffic results

For this scenario results shows us the importance of coordination between adjacent intersections, as both dynamic versions outperform the static version, but, the variant with coordination gets an even better reduction in all the metrics compared to the one without coordination. This is due to the spillback incidents, happening in the STL algorithm without coordination, when green time is wasted because vehicles can't cross the intersection due to congestion in the egress lane even though their lane is placed as green. The variation with coordination and without achieved, respectively, a reduction of 54% and 45% in trip duration, 72% and 59% in waiting time and 53% and 43% in emissions and fuel consumption.

C. Dynamic Version comparison

Table IV presents the performance metrics for the STL algorithms with and without adjacent TL coordination. This shows that enabling coordination leads to an extra reduction of 18% in trip duration, 30% in waiting time and 17% in emissions and fuel consumption in high traffic scenarios. In

TABLE IV

COMPARISON OF THE RESULTS OBTAINED IN THE SIMULATIONS OF THE DYNAMIC ALGORITHM WITH COORDINATION VERSUS THE VERSION WITHOUT COORDINATION (THE PERCENTAGES REPRESENT THE REDUCTION IN THE ANALYSED METRICS FOR THE DYNAMIC ALGORITHM WITH COORDINATION COMPARED TO THE VERSION WITHOUT COORDINATION).

	Algorithm vs STL	
	Traffic density	STL w/ coordination
Average trip duration (seconds)	Low	1%
	High	17%
Average waiting time (seconds)	Low	39%
	High	30%
Average CO2 emissions (grams)	Low	1%
	High	17%
Average fuel consumption (grams)	Low	1%
	High	17%

the low traffic scenario, the results are nearly identical, except for a 39% reduction in waiting time. While this constitutes a large percentage decrease in waiting time, the absolute values are already very low for both algorithm, both being below two seconds as we observed in Table II.

VI. CONCLUSIONS

The proposed approach was evaluated through a batch of simulation runs, using the Eclipse Mosaic framework, to assess the impact of a dynamic approach for controlling TLs and the extension to enable coordination between various intersections with TLs in an actuation area. Under the specific conditions and parameters tested, the results demonstrated that implementing dynamic schemes can significantly optimize the traffic flow at intersections compared to the static control mechanisms currently deployed at most intersections today. Additionally, the findings revealed further benefits achievable through coordination between adjacent intersections, leading to even greater traffic flow optimization and notable reductions in spillback incidents.

Future work will focus on several key areas to enhance this solution further. Firstly, the inclusion of pedestrians and the integration of pedestrian traffic flows into the control scheme will be a critical extension, ensuring a more comprehensive and inclusive approach.

In addition, future developments will focus on Enhancing the system's ability to manage traffic more efficiently while addressing current limitations, such as prioritizing public transportation, enabling emergency response, and handling accidents.

Furthermore, the system will be evaluated under realistic traffic conditions by utilizing open-source SUMO simulation scenarios. These simulations will be calibrated with real-world traffic data to enable a comprehensive assessment of the proposed approach performance in practical, real-life environments.

ACKNOWLEDGMENT

This work is supported by national funds, through the Operational Competitiveness and Internationalization Programme (COMPETE 2020) [Project nº 179491; Funding Reference: SIFN-01-9999-FN-179491].

REFERENCES

- [1] INRIX, "Scorecard," [Online]. Available: <https://inrix.com/scorecard/>. [Accessed: Aug. 23, 2024].
- [2] Federal Highway Administration (FHWA), "About Intersection Safety," [Online]. Available: <https://highways.dot.gov/safety/intersection-safety/about>. [Accessed: Aug. 23, 2024].
- [3] L. Jácome, L. Benavides, D. Jara, G. Riofrio, F. Alvarado, and M. Pesantez, "A Survey on Intelligent Traffic Lights," in *Proc. 2018 IEEE Int. Conf. on Automation/XXIII Congr. of the Chilean Assoc. of Automatic Control (ICA-ACCA)*, Concepcion, Chile, 2018, pp. 1-6. [Online]. Available: <https://doi.org/10.1109/ICA-ACCA.2018.8609705>. [Accessed: Jan. 30, 2025].
- [4] I. Tomar, I. Sreedevi, and N. Pandey, "State-of-Art Review of Traffic Light Synchronization for Intelligent Vehicles: Current Status, Challenges, and Emerging Trends," *Electronics*, vol. 11, no. 465, 2022. [Online]. Available: <https://doi.org/10.3390/electronics11030465>. [Accessed: Jan. 30, 2025].
- [5] Z. Xiao, Z. Xiao, D. Wang, and X. Li, "An Intelligent Traffic Light Control Approach for Reducing Vehicles' CO2 Emissions in VANET," in *Proc. 2015 12th Int. Conf. on Fuzzy Systems and Knowledge Discovery (FSKD)*, 2015, pp. 2070-2075. [Online]. Available: <https://doi.org/10.1109/FSKD.2015.7382270>.
- [6] M. I.-C. Wang, C. H.-P. Wen, and H. J. Chao, "Roadrunner+: An Autonomous Intersection Management Cooperating with Connected Autonomous Vehicles and Pedestrians with Spillback Considered," *ACM Trans. Cyber-Phys. Syst.*, vol. 6, no. 1, Art. no. 5, pp. 1-29, Nov. 2021. [Online]. Available: <https://doi.org/10.1145/3488246>. [Accessed: Jul. 7, 2024].
- [7] S. A. Gamel, A. I. Saleh, and H. A. Ali, "A Fog-Based Traffic Light Management Strategy (TLMS) Based on Fuzzy Inference Engine," *Neural Computing and Applications*, vol. 34, no. 3, pp. 2187-2205, 2022. [Online]. Available: <https://doi.org/10.1007/s00521-021-06525-2>.
- [8] H. Qin and H. Zhang, "Intelligent Traffic Light Under Fog Computing Platform in Data Control of Real-Time Traffic Flow," *J. Supercomput.*., vol. 77, no. 5, pp. 4461-4483, May 2021. DOI: 10.1007/s
- [9] European Telecommunications Standards Institute (ETSI), "Intelligent Transport Systems (ITS); Vehicular Communications; Basic Set of Applications; Part 2: Specification of Cooperative Awareness Basic Service," ETSI, Tech. Rep. EN 302 637-2 V1.3.1, June 2014. [Online]. Available: https://www.etsi.org/deliver/etsi_en/302600_302699/30263702/01.03.01_30/en_30263702v010301v.pdf.
- [10] TransAID, "Definition of V2X Message Sets – Deliverable 5.1," [Online]. Available: <https://www.transaid.eu/deliverables/>. [Accessed: Sep. 18, 2024].
- [11] Eclipse Foundation, "Eclipse MOSAIC: Multi-Domain and Multi-Scale Simulation Framework," [Online]. Available: <https://eclipse.dev/mosaic/>. [Accessed: Sep. 30, 2024].
- [12] Eclipse Foundation, "Eclipse SUMO: Simulation of Urban Mobility," [Online]. Available: <https://eclipse.dev/sumo/>. [Accessed: Sep. 30, 2024].
- [13] FOKUS Fraunhofer, "Simple Network Simulator (SNS)," in *Eclipse MOSAIC – A Multi-Domain and Multi-Scale Simulation Framework for Connected and Automated Mobility*, [Online]. Available: https://www.eclipse.dev/mosaic/docs/simulators/network_simulator_sns/. [Accessed: Jul. 17, 2024].
- [14] DLR, "Netedit Documentation," [Online]. Available: <https://sumo.dlr.de/docs/Netedit/index.html>. [Accessed: Jul. 30, 2024].